FitCRM

Coolify Deployment Guide

Audience: Any FitCRM/Algo Plus team member performing a deploy

Target platform: Coolify (self-hosted PaaS), Docker Compose build pack

Stack: Laravel 12 + Filament (PHP 8.2-FPM), MySQL 8.4, Redis 7

Prepared: August 2026

CONTENTS

FitCRM — Coolify Deployment Guide

A step-by-step walkthrough for deploying and operating FitCRM on Coolify, written for any team member doing the deploy for the first time.

1 — OVERVIEW

Architecture Overview

FitCRM deploys as five containers built from one image, defined in `docker-compose.yaml` at the repo root. Coolify builds the image once and runs it three times under different roles.

app WEB

- Runs PHP-FPM + nginx, serves HTTP on port 80
- Only this container runs `scripts/coolify-deploy.sh` on boot (migrations, cache warm) — the other two never race it
- Health-checked at `/healthz` (shallow) and `/up` (deep, Laravel's own check)

queue CONTAINER_ROLE=QUEUE

- Same image as `app`, different entrypoint branch
- Processes WhatsApp sends, broadcast batches, automation runs

scheduler CONTAINER_ROLE=SCHEDULER

- Runs Laravel's scheduler loop
- Fires `fitcrm:automations:resume` every 5 minutes to wake up waiting automations

mysql + redis

- MySQL 8.4 — primary datastore, persisted via a named volume
- Redis 7 — cache, session store, and queue driver

Why this matters when you deploy

All three app-image containers rebuild and restart on every deploy. Only `app` runs migrations — if you ever add a fourth replica of `app` behind a load balancer, only one instance should carry the deploy hook.

2 — BEFORE YOU START

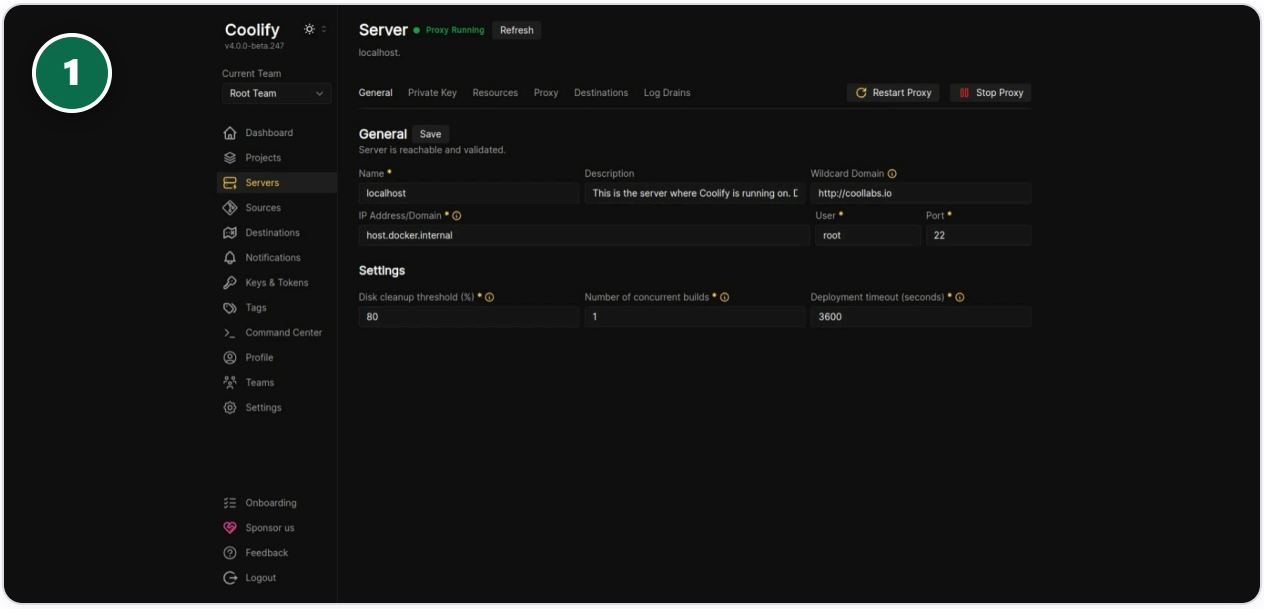
Prerequisites

Confirm each of these before opening Coolify — most deploy failures trace back to one of these being missing.

REQUIREMENT	DETAIL
A server Coolify can reach	Any VPS/cloud box with a public IP and SSH access as `root` (or a sudo user). Coolify itself may already be installed here, or you're attaching a fresh server to an existing Coolify instance.
Domain DNS	An A record for your production domain (e.g. `app.fitcrm.example.com`) pointing at the server's IP, before you request SSL in Step 3 — Let's Encrypt needs it resolvable.
Git access	Coolify needs to pull the `Fit_crm_algoplus` repo — connect it via a Deploy Key or GitHub App under **Sources** in Coolify.
A generated `APP_KEY`	Run `php artisan key:generate --show` locally (or on any PHP 8.2 box) and save the output — you'll paste it as an environment variable in Step 4.
MySQL credentials you choose	Pick a `DB_PASSWORD` now — it's used by both the `app` container and the `mysql` container to authenticate to each other.
WhatsApp / Meta credentials	`WHATSAPP_APP_SECRET` and `WHATSAPP_VERIFY_TOKEN` — Meta requires a reachable, verified webhook URL before it will deliver anything, so these should be set from the first deploy, not added later.

1 Add Your Server

Servers → + Add — reference screen shown; your own server list will be empty on a fresh Coolify instance.

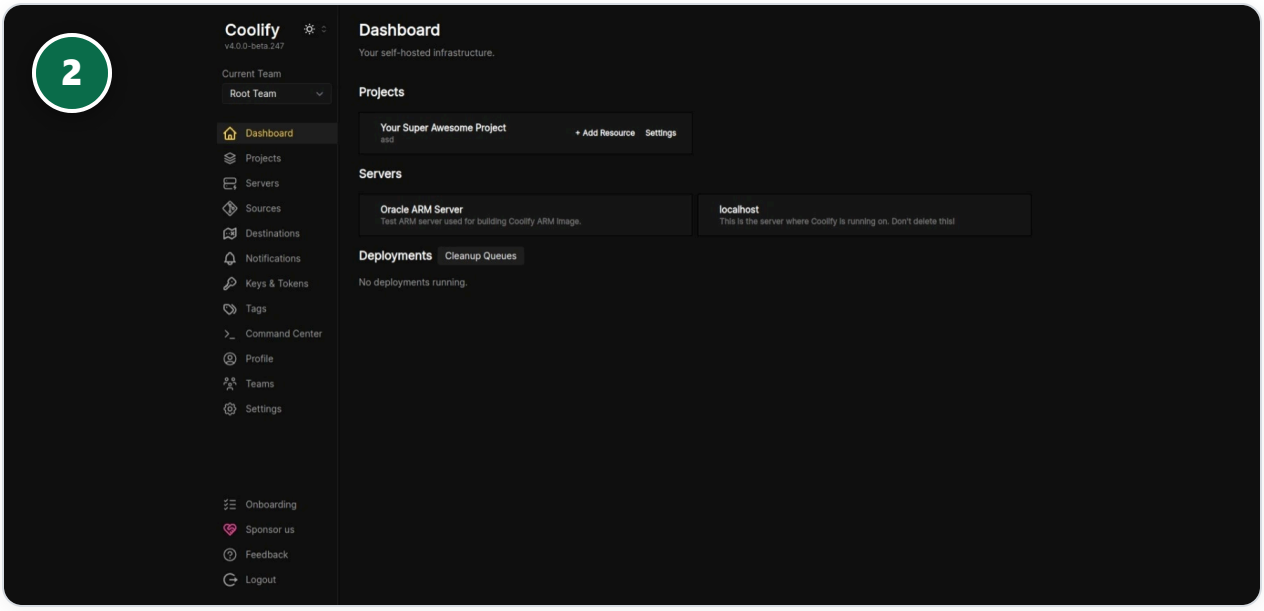


WHAT TO DO ON THIS SCREEN

1. Set **Name** to something identifiable, e.g. `fitcrm-prod`
2. **IP Address/Domain** — your VPS's public IP
3. **User** — `root` (or your sudo SSH user)
4. **Port** — `22` unless you've changed SSH's port
5. Click **Save** — Coolify validates the SSH connection immediately and shows "Server is reachable and validated."

2 Create the Project & Resource

Dashboard → your project → + Add Resource

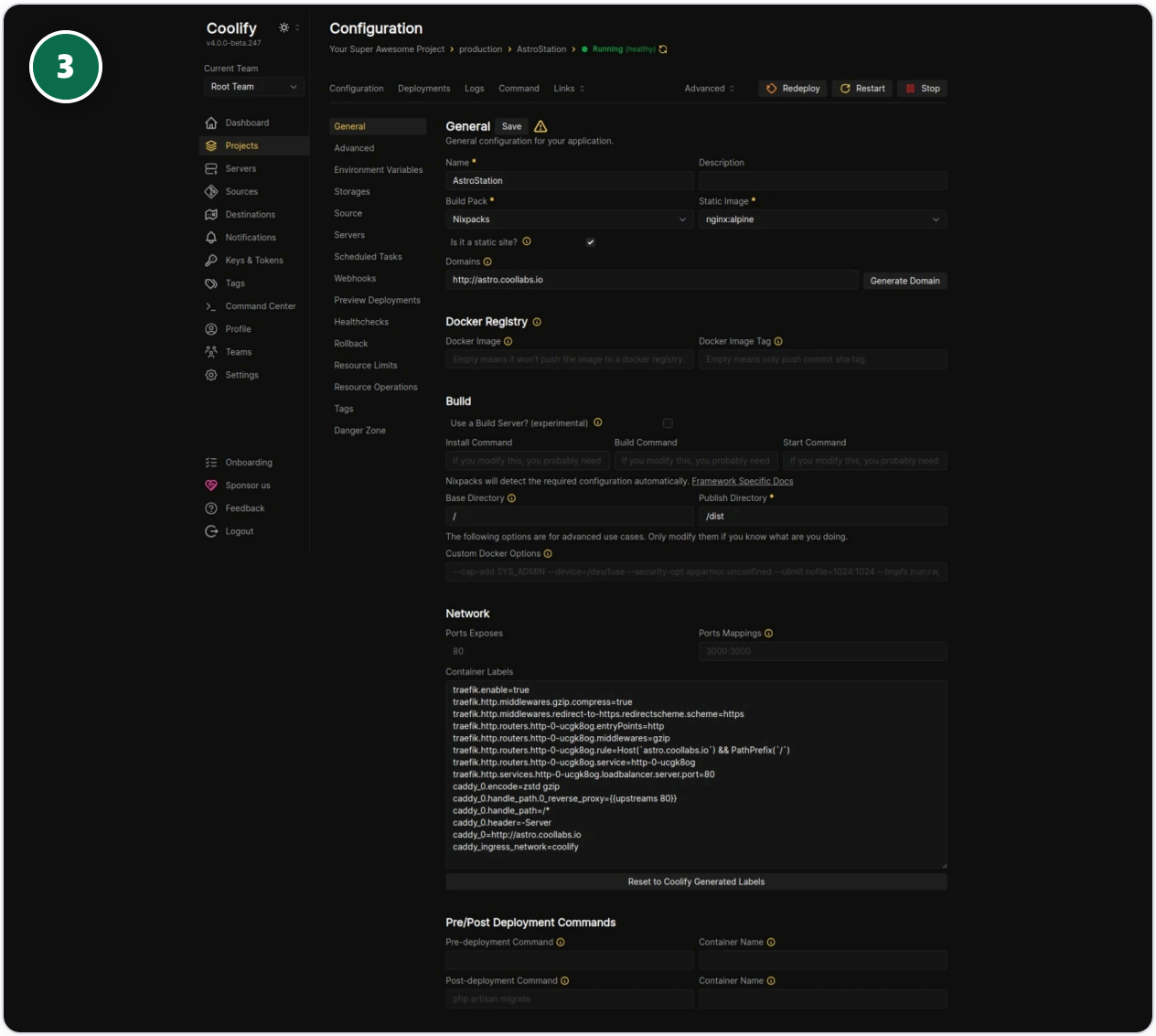


WHAT TO DO ON THIS SCREEN

1. Create a new Project named `FitCRM` (or reuse an existing one)
2. Inside it, click **+ Add Resource**
3. Choose **Docker Compose** as the resource type — not a single-app build pack. FitCRM ships five services in one `docker-compose.yaml`, so the whole stack deploys as one Coolify resource.
4. Point it at the `Fit_crm_algoplus` Git repository and the `main` branch

3 Configure the Application

Resource → Configuration — General tab. The screen shown is Coolify's single-app config (Nixpacks/Vite example) — FitCRM's own Docker Compose config page has the equivalent Repository / Domains fields.



WHAT TO DO ON THIS SCREEN

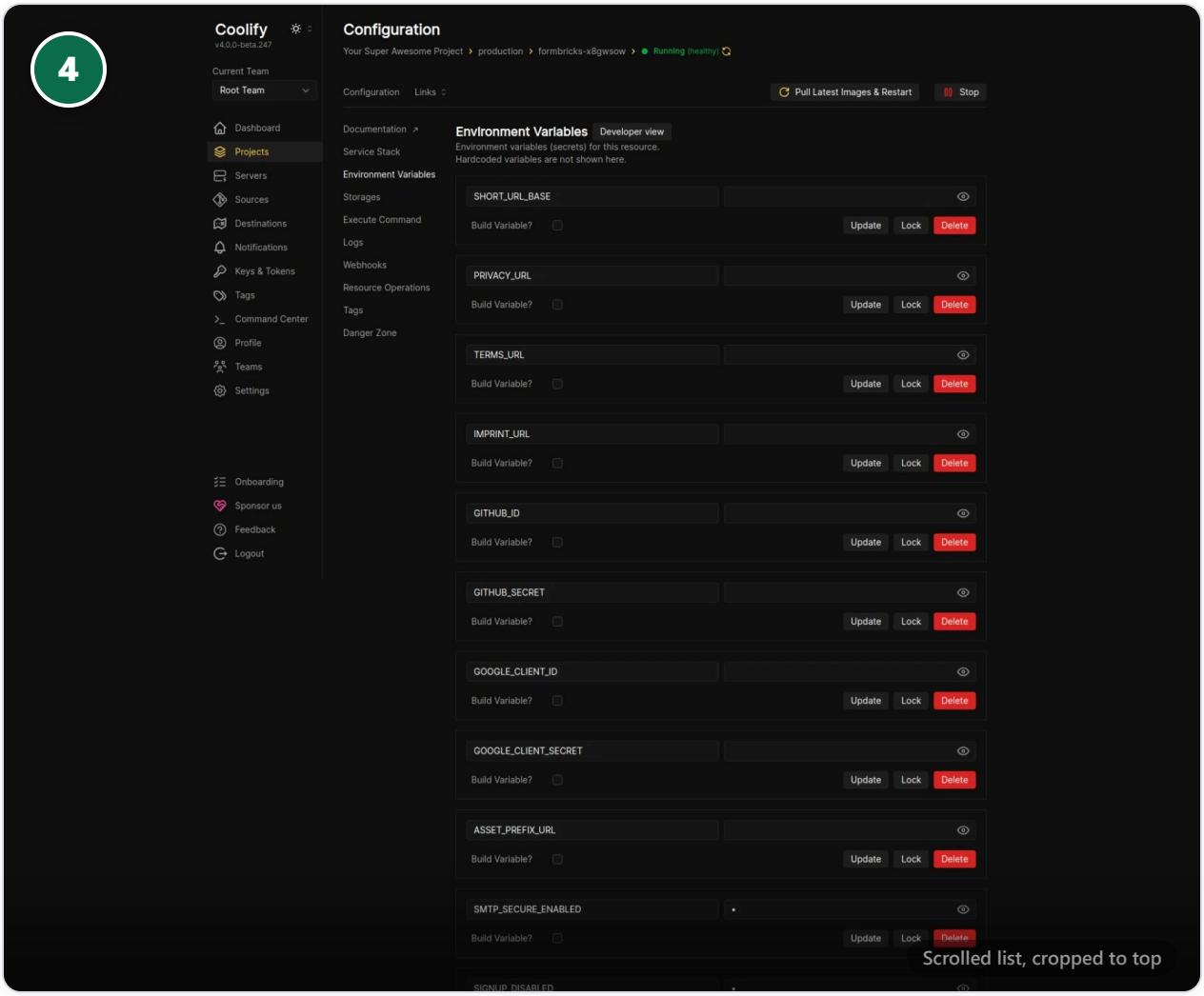
1. **Build Pack** must be `Docker Compose`
2. **Docker Compose Location** — `docker-compose.yaml` (repo root — this is the default, just confirm it)
3. **Domains** — enter your production domain, e.g. `https://app.fitcrm.example.com`, then click **Generate Domain**/**Save** so Coolify provisions the Let's Encrypt certificate
4. Leave **Force HTTPS** on — `SESSION_SECURE_COOKIE=true` in FitCRM's config assumes it

---

Note: this reference screenshot is from Coolify's own documentation and shows a single-app (Nixpacks) example, not FitCRM's actual Compose config screen — the field names and location in the UI are the same, the values shown in the boxes are not FitCRM's.

4 Set Environment Variables

Resource → Environment Variables — paste every key from .env.example with real production values.



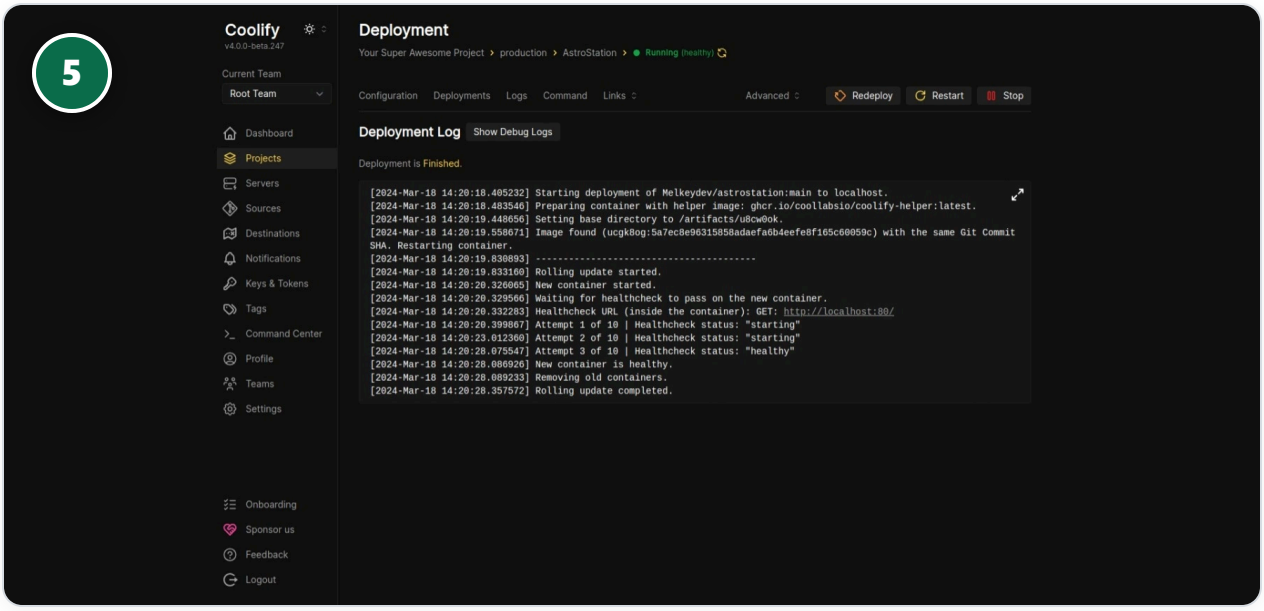
WHAT TO DO ON THIS SCREEN

1. **APP_KEY** — Output of `php artisan key:generate --show` (Prerequisites)
2. **APP_URL** — e.g. `https://app.fitcrm.example.com` — must match the domain from Step 3
3. **APP_DEBUG** — `false` — never `true` in production
4. **TRUSTED_PROXIES** — `*` — required behind Coolify's Traefik proxy for correct HTTPS/secure-cookie behavior
5. **DB_HOST / DB_DATABASE / DB_USERNAME / DB_PASSWORD** — Point at the `mysql` service name; password matches what you chose in Prerequisites
6. **REDIS_HOST** — `redis` — the Compose service name, not an IP
7. **QUEUE_CONNECTION / CACHE_STORE** — `redis` for both
8. **WHATSAPP_APP_SECRET / WHATSAPP_VERIFY_TOKEN** — From your Meta App dashboard — required before Meta will call the webhook
9. **MAIL_*** — Your SMTP provider, or leave `MAIL_MAILER=log` to defer email until later

Reference screenshot shows another project's variable list (from Coolify's docs) to illustrate the screen layout — the key names shown in it are not FitCRM's. Use FitCRM's own `.env.example` in the repo root as the authoritative source of which keys to set.

5 Deploy

Click Deploy (top right) and watch the log stream live.

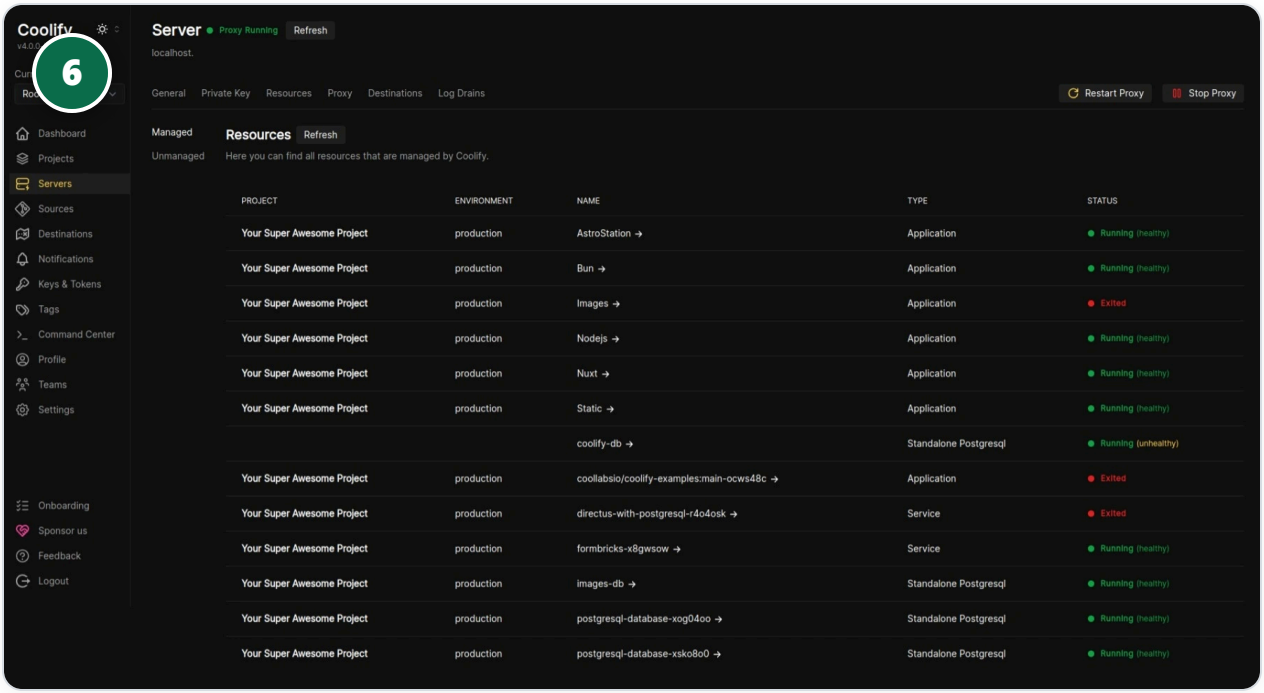


WHAT TO DO ON THIS SCREEN

1. Click **Deploy** — Coolify builds the image from the `Dockerfile`'s multi-stage build (Node stage compiles Vite assets, then a PHP-FPM + nginx runtime stage)
2. Watch for `scripts/coolify-deploy.sh` running inside the **app** container: `migrate --force` → `config:cache` → `route:cache` → `view:cache` → `filament:optimize`
3. Coolify waits for the `/healthz` healthcheck to report **healthy** before routing traffic to the new container — this is what "Rolling update completed" confirms in the log, same pattern shown here
4. If the deploy fails at the migration step, check **Known Issues** at the end of this guide before re-running

6 Verify All Services Are Healthy

Server → Resources tab — confirm every FitCRM container shows Running (healthy).

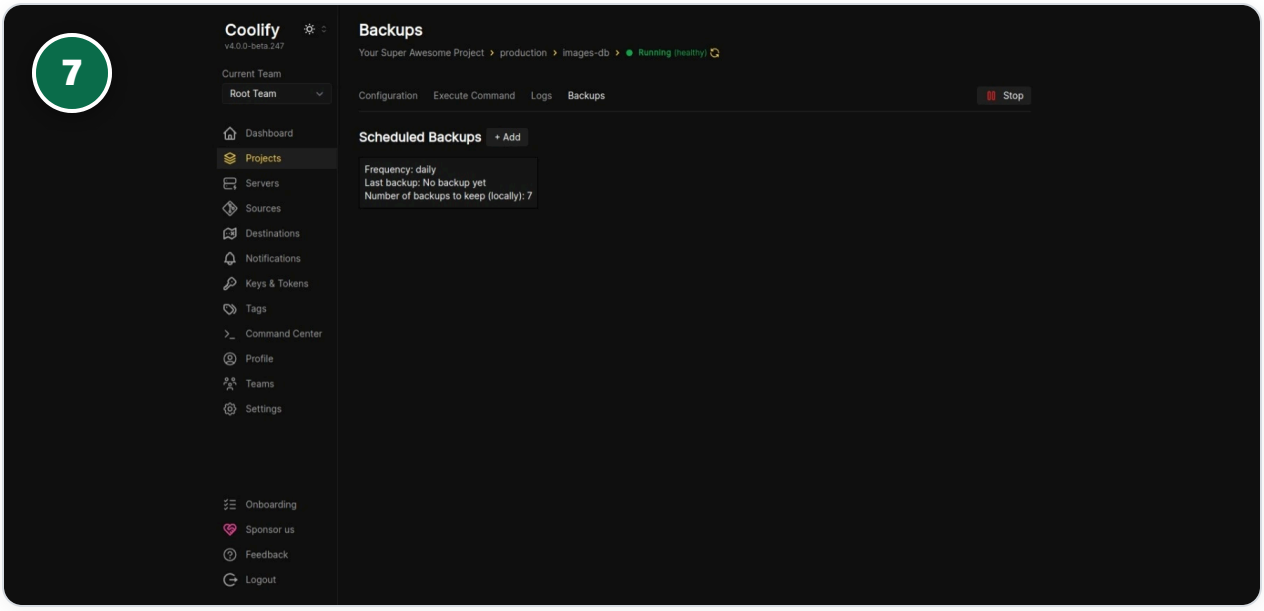


WHAT TO DO ON THIS SCREEN

1. You should see five entries for this resource: **app**, **queue**, **scheduler**, **mysql**, **redis**
2. All five should read **Running (healthy)** — `app` and `mysql` have explicit Docker healthchecks; `queue` / `scheduler` show healthy once they start without crash-looping
3. If `queue` or `scheduler` restart repeatedly, check their logs — the most common cause is `mysql` not yet accepting connections when they first booted (see Known Issues)
4. From your own machine, confirm both health routes respond: `curl -f https://your-domain/healthz` and `curl -f https://your-domain/up`

7 Enable Scheduled Database Backups

Resource (mysql) → Backups tab.

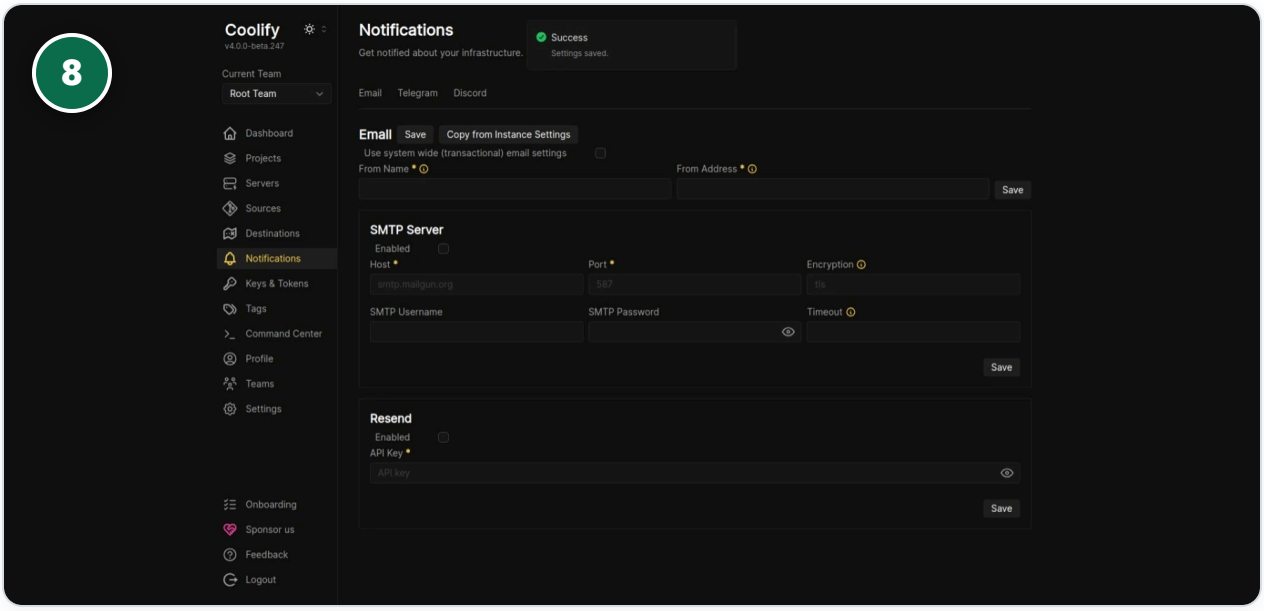


WHAT TO DO ON THIS SCREEN

1. Click **+ Add** under Scheduled Backups
2. Set **Frequency** to `daily` at minimum for production
3. Set **Number of backups to keep** — 7 is a reasonable starting point
4. Optionally enable **S3 Enabled** (visible on the Configuration sub-tab) to ship backups off-server — strongly recommended once real member/biometric data is in the database

8 Set Up Deploy Notifications

Team → Notifications.



WHAT TO DO ON THIS SCREEN

1. Enable at least one channel — **Email**, **Discord**, or **Telegram** — so the team knows immediately if a deploy or a container healthcheck fails
2. This is what will alert you if, for example, the `anthropic-ai/sdk` composer-lock issue (see Known Issues) causes a runtime error after a deploy that otherwise looked successful

POST-DEPLOY

Post-Deploy Verification Checklist

Run through this after every deploy, not just the first one.

CHECK	HOW
App responds	`curl -f https://your-domain/healthz` and `/up` both return 200
Login works	Log into the Filament admin panel with a known superadmin account
Queue is processing	`docker compose logs queue` — no repeated crash/restart lines
Scheduler is ticking	`docker compose logs scheduler` — should show a run roughly every minute
WhatsApp webhook verified	Meta App dashboard → Webhooks shows a green "Verified" status against your `/api/webhooks/whatsapp` URL
Migrations applied	`docker compose exec app php artisan migrate:status` — no pending migrations

READ BEFORE YOU DEPLOY

Known Issues to Fix Before Go-Live

Found during a static production-readiness audit of this codebase (August 2026). Not deploy blockers for a first test deploy, but the first item will cause a real runtime error once live.

CRITICAL — composer.lock is missing the AI reply assistant's dependency

`composer.json` requires `anthropic-ai/sdk`, but it is entirely absent from `composer.lock`. The Docker build will succeed and the app will boot fine — but the first time anyone clicks "AI Suggest Reply" in the WhatsApp inbox, or saves the AI Assistant settings, it will throw a fatal `Class not found` error in production.

Fix before deploying:

run `composer update anthropic-ai/sdk` locally, commit the regenerated `composer.lock`, then deploy.

MEDIUM — Docker Compose health-checks aren't gated on readiness

`mysql` has a healthcheck, but nothing in `app` / `queue` / `scheduler`'s `depends_on` waits for it to pass (only for the container to start). On a cold first boot, `app` can try to migrate before MySQL is accepting connections, causing a crash-restart loop until it catches up. It self-heals via `restart: unless-stopped`, but expect a noisy first deploy.

HIGH (security) — SSRF risk in the WhatsApp automation "webhook" step

Any branch-level staff user with access to the Automations builder can configure a webhook step pointing at an internal address (e.g. the Docker network's `mysql` / `redis` hosts, or a cloud metadata IP) — there's no denylist on the configured URL. Recommended fix: reject private/loopback/link-local targets in `AutomationStepExecutor::callWebhook()` before this goes in front of real branch-admin users.

Validate the fixes with these commands on a machine with PHP 8.2 + Composer + Docker before your first production deploy:

```
composer update anthropic-ai/sdk
composer install
php artisan test
sh -n docker/entrypoint.sh && sh -n scripts/coolify-deploy.sh
docker compose config
docker build -t fitcrm-test .
docker compose up -d --build
curl -f http://localhost/healthz && curl -f http://localhost/up
docker compose logs queue scheduler
```